GPU-Accelerated Discrete Element Method (DEM) Simulation of a Snowball Avalanche Dynamics on NVIDIA Maxwell

Maurizio Grisotto
Politecnico di Torino
Turin, Italy 2026
s336145@studenti.polito.it

***Abstract*—We present a real-time simulation of a growing snowball rolling down an inclined slope, capturing static snowpack particles via a Discrete Element Method (DEM)-inspired contact adhesion model as a potential component of a real-time game graphics engine. This paper investigates the concepts covered in the "GPU Programming" course through the implementation of a snowball simulation with 3 different applications: CPU Naïve, CPU-Optimized, and GPU. Both the algorithmic/performance aspects and dynamic Vulkan rendering (GPU version only) were addressed. A theoretical analysis of the multi-GPU extension scenario is also provided. The simulation is implemented entirely in CUDA targeting NVIDIA Maxwell GPUs (SM 5.0/5.3) and features a multi-stream pipeline with adaptive kernel fusion, warp-cooperative, shared-memory tiling, lock-free atomic capture, spatial hashing with CUB radix sort, and zero-spill register allocation across all kernels. We detail each optimization with quantitative analysis of occupancy, memory bandwidth, and launch overhead trade-offs. The GPU implementation is benchmarked against scalar (SISD) and SSE/NEON + OpenMP (SIMD) CPU baselines on a laptop GTX 960M and a Jetson Nano, demonstrating the effectiveness of architecture-aware CUDA programming on resource-constrained Maxwell hardware.**

***Index Terms*—CUDA, GPU programming, Discrete Element Method, particle simulation, Maxwell architecture, shared memory tiling, spatial hashing, kernel fusion, occupancy tuning**

I Introduction

Granular-material simulations are central to computational physics and engineering, with applications ranging from geotechnical hazard prediction to visual effects in games and films. The *Discrete Element Method* (DEM) [1] models granular media as collections of individual particles interacting through pairwise contact forces, providing a physically faithful representation of phenomena such as avalanches, landslides, and snowfall accumulation. However, DEM simulations are computationally demanding: at each timestep every particle must detect contacts with its neighbours and resolve penalty-based forces, yielding an $O(N \cdot k)$ cost per frame (where k is the average neighbour count) that quickly exceeds the real-time budget of a single CPU core.

Graphics Processing Units (GPUs), with their thousands of lightweight cores and high-bandwidth memory, are a natural fit for DEM workloads. Each particle can be mapped to an independent CUDA thread, and the Structure-of-Arrays (SoA) memory layout ensures coalesced global-memory transactions. Several works have demonstrated GPU-accelerated particle simulations [2], [3], yet most target high-end datacenter GPUs. Optimizing for *resource-constrained* Maxwell-class hardware (5 Streaming Multiprocessors, 48 KB shared memory, 64 K registers per SM) introduces additional challenges: every register counts, shared-memory budgets are tight, and kernel launch overhead is a measurable fraction of frame time.

A. The System

This paper presents **Angry Santa**, a real-time DEM snowball-avalanche simulator implemented in CUDA and benchmarked on two Maxwell-class devices: a laptop NVIDIA GTX 960M (SM 5.0) and an NVIDIA Jetson Nano (SM 5.3). The main components are:

1) A **three-tier particle architecture** (static snowpack, dynamic shell, rigid-body core) with a probabilistic sigmoid-based capture model that converts snowpack particles into shell particles as the snowball rolls.
2) A **multi-stream GPU pipeline** with fork/join synchronization and an **adaptive kernel-fusion strategy** that switches between a grid-based $O(N \cdot k)$ collision path and a single mega-fused $O(N^2)$ kernel depending on the active particle count.
3) A comprehensive set of **architecture-aware optimizations** — warp-cooperative shared-memory tiling, lock-free atomic capture, zero-spill register allocation, gated event recording, sorted-snowpack binary-search range limiting — each analysed with occupancy and bandwidth trade-offs specific to Maxwell.
4) **Scalar (SISD) and SIMD + OpenMP CPU baselines** that share the same physics pipeline, enabling direct speedup measurements.

B. Paper Organisation

Section II describes the DEM physics model, terrain geometry, and capture algorithm. Section III details the three-tier data architecture and per-frame data flow. Section IV presents the dual-path GPU pipeline. Section V is the core of the paper, covering all GPU optimizations with quantitative analysis. Section VI summarises the CPU reference implementations. Section VII reports experimental results, and Section VIII concludes.

II Physics Model

The simulation employs a **soft-sphere DEM** contact model (penalty-based) with Coulomb friction and *without* persistent contact history: tangential springs are not tracked across frames, and the friction impulse is recomputed independently at each timestep from the current relative velocity.

A. Terrain Geometry

The simulation takes place on an infinite inclined plane characterised by slope angle θ (default 30°) and height offset $H = 30\,\text{m}$:

$$\sin\theta \cdot x + \cos\theta \cdot y = H. \tag{1}$$

The slope normal is $\hat{N} = (\sin\theta,\ \cos\theta,\ 0)$ and the downhill tangent is $\hat{T} = (\cos\theta,\ -\sin\theta,\ 0)$. All particle positions are expressed in world coordinates; a curvilinear parameterisation (s, n) maps to world as:

$$x = s\cos\theta + n\sin\theta, \quad y = \frac{H}{\cos\theta} - s\sin\theta + n\cos\theta, \tag{2}$$

where s is the arc-length along the slope and n the surface-normal offset.

B. Snowpack Initialisation

One million particles are placed on a rectangular band parameterised by (s, z, n): $s \in [s_0,\ s_0 + L_s]$ (downhill), $z \in [-W/2,\ +W/2]$ (lateral), $n \in [0,\ \text{thickness}]$ (normal layers). A regular grid is constructed within each layer and each particle receives $\pm 30\%$ jitter to break regularity. Per-particle wetness $w \in [w_{\min}, w_{\max}]$ is assigned randomly via `curand` (called once at initialisation). After placement, the snowpack is **sorted once by ascending `posX`** using CUB radix sort, enabling per-frame binary-search range limiting (Section V-J).

C. Snowball Core — Rigid Body

The snowball rolls under gravity along the slope with downhill acceleration $a_{\text{slope}} = g\sin\theta$. Two drag forces oppose the motion:

1) **Rolling drag** (linear): $\mathbf{v} \leftarrow \mathbf{v} \cdot (1 - k_{\text{roll}} \cdot \Delta t)$.
2) **Aerodynamic drag** (quadratic, proportional to frontal area R^2):

$$\Delta v_{\text{aero}} = \frac{C_{\text{aero}} \cdot R^2 \cdot |\mathbf{v}|}{m}\,\Delta t, \tag{3}$$

applied opposite to velocity and clamped to prevent motion reversal.

The ball is constrained to the slope surface: if the signed distance $d = \sin\theta \cdot x + \cos\theta \cdot y - H$ falls below R, the ball is pushed out along $\hat{N}$ and the normal velocity component is zeroed.

Rolling-without-slip angular velocity is:

$$\boldsymbol{\omega} = \frac{1}{R}\,(\hat{N}_{\text{slope}} \times \mathbf{v}). \tag{4}$$

Orientation is maintained via **quaternion integration**: $\mathbf{q}' = \mathbf{q} + \frac{1}{2}\Delta t\ (0, \boldsymbol{\omega}) \otimes \mathbf{q}$, with renormalisation each frame.

D. Probabilistic Capture

The capture algorithm (Fig. 1) converts snowpack particles into dynamic shell particles. Each frame, the host performs a binary search on the sorted `posX` array to identify the relevant window $[\text{lo}, \text{hi})$; the kernel processes only this range. For each candidate particle within capture radius $r_{\text{cap}} = R + r_p + 0.3\,\text{m}$:

1) Compute a **logit** score:

$$\ell = k_0 + k_1 \cdot w - k_2 \cdot |v_{\text{rel}}| + k_R \cdot R, \tag{5}$$

where the $k_R \cdot R$ term provides *avalanche feedback* (larger ball → easier capture). The logit is clamped to $[-15, +15]$.
2) Apply the **sigmoid**: $P_{\text{stick}} = \sigma(\ell) = 1/(1 + e^{-\ell})$.
3) Generate a pseudo-random number via a **deterministic murmurhash** variant (seeded with particle index and frame number — not `curand`, avoiding per-thread state overhead).
4) If accepted, reserve a slot via **lock-free `atomicAdd`** on a global counter, subject to a per-frame cap (`maxCapturePerFrm` = 150) and shell capacity check.
5) Write position (projected onto ball surface), velocity (synchronised with surface point via $\boldsymbol{\omega} \times \mathbf{r}$), and body-frame attachment direction (via inverse quaternion rotation).

Three stability mechanisms prevent runaway growth: logit clamping, the per-frame capture cap, and proportional mass scaling when the cap is exceeded (Section III).

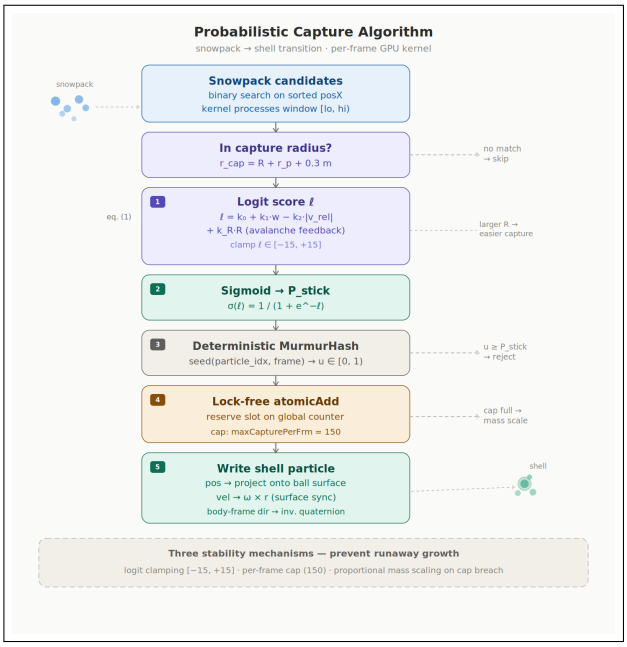

Fig. 1: The probabilistic capture algorithm overview. A binary search narrows the launch domain; each thread evaluates a sigmoid probability and reserves a shell slot via `atomicAdd` increasing the capacity.

E. Shell Particle Forces

Each shell particle accumulates four force contributions:

1) **Gravity**: $\mathbf{F}_g = m\,(0, -g, 0)$.
2) **Mass-proportional damping**: $\mathbf{F}_d = -k_d \cdot m \cdot \mathbf{v}$.
3) **Tether spring-damper** to the ball surface anchor:

$$\mathbf{F}_{\text{tether}} = K(\mathbf{x}_{\text{tgt}} - \mathbf{x}) + D(\mathbf{v}_{\text{tgt}} - \mathbf{v}), \tag{6}$$

   where the target position is computed from the ball's quaternion and each particle's body-frame attachment direction.
4) **DEM contact** (computed in a separate collision kernel): soft-sphere penalty with Coulomb tangential friction ($|\mathbf{F}_t| \leq \mu_p |\mathbf{F}_n|$) and short-range cohesion for overlapping pairs.

F. Integration and Ground Collision

A **semi-implicit (symplectic) Euler** scheme updates velocity then position:

$$\mathbf{v}^{n+1} = \mathbf{v}^n + \frac{\mathbf{F}}{m}\Delta t, \qquad \mathbf{x}^{n+1} = \mathbf{x}^n + \mathbf{v}^{n+1}\Delta t. \tag{7}$$

Ground collision resolves penetration against the inclined plane with normal restitution ($e = 0.15$) and Coulomb friction ($\mu = 0.14$). When particles are captured, the ball velocity is scaled to conserve linear momentum: $\mathbf{v}' = \frac{m_{\text{old}}}{m_{\text{old}}+\Delta m}\,\mathbf{v}$.

III System Architecture

The simulation state is partitioned into three cooperating subsystems, each with distinct mutability semantics (Fig. 2).

A. Snowpack (Static)

The snowpack is a read-mostly Structure-of-Arrays (SoA) containing 10^6 particles with positions, radii, masses, wetness values, and an `alive` flag (`AVAILABLE`/`CONSUMED`). Positions are immutable after initialisation; only the `alive` flag is toggled atomically upon capture. The array is sorted once by `posX` to enable binary-search range limiting.

B. Shell (Dynamic)

The shell is a fully dynamic SoA (`ParticleSystem`) storing positions, velocities, forces, masses, radii, state flags, wetness, and body-frame attachment vectors (`attachLocal`). Its capacity starts at 256 and **grows dynamically** (doubling) as particles are captured, using `cudaMalloc`/`cudaMemcpy` with the old buffer freed after copy. Under steady-state conditions (1M snowpack, default parameters), the shell stabilises at approximately 4,000 active particles.

C. Core (Rigid Body)

The snowball core is a single `SnowballState` struct maintained on the host: position, velocity, quaternion orientation, angular velocity, mass, and radius. It is updated each frame via rigid-body slope dynamics with rolling and aerodynamic drag. Mass and radius grow as captured particles contribute mass, with the radius derived as $R = (3m/4\pi\rho)^{1/3}$.

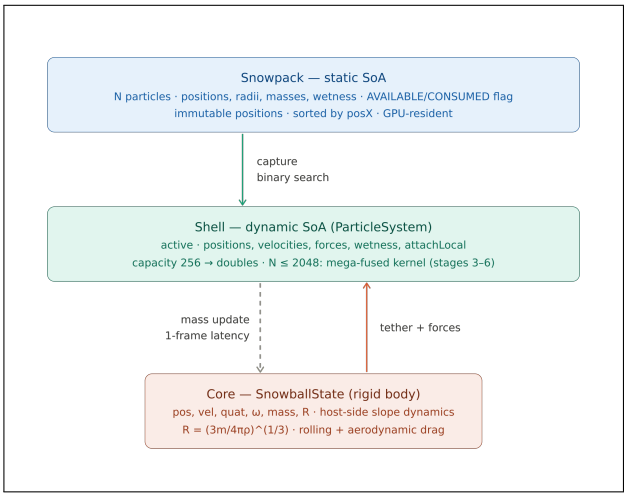

Fig. 2: Three-tier particle architecture. The capture kernel converts snowpack particles into shell particles; each shell particle is tethered to the rigid-body core via a spring-damper.

D. Per-Frame Data Flow

The per-frame pipeline proceeds as follows:

1) **Capture**: snowpack → shell (range-limited binary search).
2) **Core mass update**: deferred readback (1-frame latency) of capture counters from device; momentum-conserving velocity scaling.
3) **Forces + Tether**: gravity, damping, and tether spring (fused kernel).
4) **Grid build**: spatial hash, CUB radix sort, cell ranges (*parallel stream*).
5) **Collision**: neighbour DEM + cohesion.
6) **Integrate + Ground**: semi-implicit Euler + slope constraint (fused kernel).
7) **Core rigid-body update**: host-side slope dynamics.

For small shell counts ($N \leq 2048$), stages 3–6 are replaced by a single **mega-fused kernel** that performs the entire physics pipeline in one launch (Section V-K).

E. Deferred Capture Readback

Capture counters (particle count and accumulated mass) reside in a packed 8-byte device buffer, reset each frame via `cudaMemsetAsync`. The result is read back asynchronously to pinned host memory. The host processes the readback at the top of the *next* frame (1-frame latency, physically negligible at 300 Hz). When more threads passed the probabilistic test than the per-frame cap allows, the credited mass is scaled proportionally:

$$m_{\text{credited}} = m_{\text{total}} \cdot \frac{\text{actualNew}}{\text{d\_captureCount}}, \tag{8}$$

preventing the ball radius from inflating and triggering a runaway feedback loop.

IV GPU Pipeline

The GPU implementation provides two execution paths selected at runtime based on the current shell particle count N. Both paths share the capture stage and the host-side core update; they differ in how shell physics is dispatched.

A. Large-N Path ($N > 2048$)

Seven stages execute on two CUDA streams with fork/join synchronisation via lightweight events (`cudaEventDisableTiming`):

TABLE I: Large-N GPU pipeline stages.

#	Stage	Kernel	Stream	Block
1	Capture	`captureFromSnowpack`	sim	256
2	Core update	host code	—	—
3	Forces+Tether	`forceTether`	sim	256
4	Grid build	hash→sort→ranges	grid	256
5	Collision	`neighborCollision`	sim	512
6	Integrate+Ground	`integrateGround`	sim	256
7	Core RB	host code	—	—

Stages 3 (forces + tether) and 4 (grid build) execute **concurrently** on `simStream` and `gridStream`, respectively. The collision kernel (stage 5) waits for both via `cudaStreamWaitEvent` on `evGridDone`. Stream synchronisation events use `cudaEventDisableTiming` to eliminate timing overhead (Section V-M).

B. Small-N Path ($N \leq 2048$)

When the shell count is small, the overhead of the grid build (5–8 GPU commands: clear + hash + CUB sort + cell ranges) exceeds the computational benefit. A single **mega-fused kernel** (`shellPhysicsFused`) replaces stages 3–6, reducing GPU commands per frame from ∼10 to **1** and keeping all intermediate forces in registers (Section V-K).

C. Block Sizes and `__launch_bounds__`

Block sizes are chosen to balance **SM coverage** (all SMs receive at least one block) against per-block resource consumption. With $N = 4{,}000$ and 5 SMs on the GTX 960M, a block of 1024 yields only $\lceil 4000/1024 \rceil = 4$ blocks, leaving 1 SM idle (20% wasted). A block of 256 yields 16 blocks, ensuring full coverage.

The `neighborCollisionKernel` uses block size 512 dictated by the warp-cooperative tile layout (MAX_WARPS $= 16 = 512/32$). All other physics kernels use 256. Each kernel is annotated with `__launch_bounds__(maxThreads, minBlocks)` to guide the register allocator (Table II).

D. Adaptive Path Selection

The $N = 2048$ threshold is chosen empirically: below this value, the mega-fused kernel's reduced command overhead (∼100–150 μs) outweighs the cost of brute-force $O(N^2)$ collision detection. Above 2048, the grid-based $O(N \cdot k)$ path becomes more efficient despite the additional GPU commands. At typical steady-state conditions ($N \approx 4{,}000$ particles), the grid path executes ∼3–4 neighbours per particle, yielding ∼16,000 pair evaluations compared to ∼ 16M in brute-force — a 1000× reduction that justifies the grid-build overhead.

V GPU Optimizations

This section details every GPU-level optimisation applied in the project, with quantitative analysis of occupancy, bandwidth, and launch-overhead trade-offs on Maxwell hardware (SM 5.0/5.3).

A. Memory Coalescing via SoA Layout

A warp of 32 threads issues a single memory transaction. With the Structure-of-Arrays layout, threads 0–31 access contiguous floats and the hardware merges the request into a single 128-byte transaction (100% efficiency). An Array-of-Structures layout would scatter accesses across a stride equal to the struct size, wasting up to 90% of bus bandwidth.

Indirect accesses introduced by the spatial grid (`posX[particleIndex[s]]`) are not perfectly coalesced. However, after CUB radix sort, particles within the same cell have nearby indices — loads are nearly coalesced and benefit from L2 cache reuse (the steady-state working set of 4,000 particles fits in ∼128 KB, or 12.5% of the 1 MB L2 on the GTX 960M).

B. Warp-Cooperative Shared-Memory Tiling

Shared memory is on-chip SRAM (∼5-cycle latency, ∼1.3 TB/s), compared to global memory (∼300–500 cycles, ∼30 GB/s on Maxwell). The `neighborCollisionKernel` employs **warp-cooperative tiling**: each warp loads a tile of 32 neighbours into shared memory and all 32 threads read from it. The tile is declared as `[MAX_WARPS][WARP_SIZE + 1]`, where MAX_WARPS $= 16 = 512/32$ and the `+1` padding avoids shared-memory bank conflicts (successive warps access different banks):

```
__shared__ float s_px[MAX_WARPS][WARP_SIZE+1];
__shared__ float s_py[MAX_WARPS][WARP_SIZE+1];
__shared__ float s_pz[MAX_WARPS][WARP_SIZE+1];
__shared__ float s_r [MAX_WARPS][WARP_SIZE+1];
__shared__ int   s_idx[MAX_WARPS][WARP_SIZE+1];
__shared__ int   s_state[MAX_WARPS][WARP_SIZE+1];
```

Why `__syncwarp()` instead of `__syncthreads()`. The previous kernel used `__syncthreads()` combined with symmetric iteration. When excess threads executed an early `return` before the barrier, the remaining threads deadlocked — violating the CUDA specification that *all* threads in a block must reach the same `__syncthreads()`. The warp-scoped `__syncwarp()` avoids this: all 32 threads always participate in the tile load (the `lane < tileLen` guard is a predicated branch, not an early return).

Why velocities are not tiled. Adding `s_vx/vy/vz` would increase per-block shared memory from ∼10.6 KB to ∼17 KB. At 3 resident blocks this exceeds the 48 KB limit (51 KB), reducing occupancy from 75% to 50%. Velocities are instead loaded on-demand via `__ldg()` only when contact is detected, which is rare (∼2–5 contacts out of 10–50 neighbours per cell).

C. Atomic Reduction Strategy

On Maxwell, `atomicAdd` on FP32 generates a compare-and-swap (CAS) loop that under contention executes 5–10 retry iterations. The previous symmetric collision kernel (`j > i`) required 6 `atomicAdd` per pair.

The current design uses **full non-symmetric iteration**: every thread i iterates over all neighbours $j \neq i$, accumulating forces entirely in registers. The sole write is a direct store to `forceX/Y/Z[i]` — thread i is the only writer, so no atomic is needed. This trades doubled FLOPs (N vs. $N/2$ pair evaluations) for **zero contention**, which is a net win on Maxwell where each CAS retry costs 20–60 cycles versus $\sim$4 cycles for a direct store.

D. Occupancy Tuning

Occupancy is defined as the ratio of active warps to the maximum warps per SM (64 on SM 5.0/5.3, i.e. 2048 threads). It is constrained by registers (65,536/SM), shared memory (48 KB/SM), and block count (max 32/SM). Table II reports the per-kernel occupancy achieved.

TABLE II: Per-kernel occupancy (`ptxas -v` output).

Kernel	Regs	Block	Blk/SM	Occ.	Limiter
`neighborCollision`	40	512	3	75.0%	Shared mem.
`forceTether`	27	256	8	100%	—
`integrateGround`	24	256	8	100%	—
`captureFromSnowpack`	48	256	5	62.5%	Registers
`shellPhysicsFused`	37	256	5	62.5%	Registers
`bruteForceCollision`	38	256	5	62.5%	Registers
`computeHash`	14	256	8	100%	—
`buildCellRanges`	7	256	8	100%	—
`clearPrevCells`	8	256	8	100%	—

All kernels report **zero register spill** in the `ptxas` output. Four kernels operate below 100%:

- **`neighborCollision` (75%)**: dual constraint — 40 regs $\times$ 512 = 20,480 regs/block ($\lfloor 65536/20480 \rfloor = 3$ blocks) and shared memory: $3 \times 12.4\,\text{KB} = 37\,\text{KB}$ fits, but $4 \times 12.4\,\text{KB} = 49.6\,\text{KB}$ exceeds 48 KB.
- **`captureFromSnowpack` (62.5%)**: 48 regs — the highest in the project. The kernel computes `expf`, hash PRNG, 3D distance, and quaternion rotation. Reaching 100% would require 32 regs (a 33% cut), causing massive spill in a kernel that is already range-limited to $\sim$500–2,000 threads per frame.
- **`shellPhysicsFused` / `bruteForceCollision` (62.5%)**: 37–38 regs + shared memory for brute-force tiling ($8 \times 257 \times 4\,\text{B} \approx 8\,\text{KB}$/block). At 6+ blocks shared memory exceeds budget. These kernels run only for $N \leq 2048$.

Design principle: zero spill $>$ 100% occupancy. A single register spill costs $\sim$300 cycles (DRAM round-trip), while one fewer warp costs only a few scheduling cycles. For compute-bound kernels or kernels whose working set fits in cache, keeping all variables in registers outweighs the benefit of additional concurrent warps.

E. Single Precision and Fast Math

Consumer Maxwell GPUs have an FP64 : FP32 throughput ratio of 1 : 32. With $\Delta t = 5\,\text{ms}$, the Euler truncation error ($O(\Delta t^2) \approx 2.5 \times 10^{-5}$) dominates FP32 rounding ($\epsilon \approx 6 \times 10^{-8}$) by three orders of magnitude, making single precision entirely adequate. All literals are `f`-suffixed, all math functions use `f`-variants (`sqrtf`, `expf`, `fmaxf`), and `M_PI` is defined as `float`. The `--use_fast_math` flag enables fused operations and flush-to-zero for denormals.

F. `__restrict__` and `__ldg()`

The `__restrict__` qualifier enables the compiler to reorder loads/stores and cache values in registers. On SM $\geq$ 3.5, `__ldg()` routes loads through the read-only texture cache, which is physically separate from the L1/shared memory used for read-write data. This **doubles effective cache bandwidth** without conflicts. All physics and grid kernels use explicit `__ldg()` for immutable inputs; the capture kernel uses `const __restrict__` pointers, which the compiler auto-routes through the read-only cache at `-O3`.

G. Constant Memory

The `SimParams` struct ($\sim$152 B) is placed in `__constant__` memory. Its dedicated 64 KB cache **broadcasts** a single value to all 32 threads in a warp in $\sim$5 cycles, without consuming registers. Physics constants (gravity, stiffness, damping, etc.) are read identically by all threads — a perfect broadcast use-case. The struct is uploaded once at startup via `cudaMemcpyToSymbol`.

H. Kernel Fusion

Two pairs of kernels were fused to eliminate global-memory round-trips:

- `applyForces` + `shellCoreTether` $\rightarrow$ **`forceTether`**: eliminates one round-trip for `forceX/Y/Z`.
- `integrate` + `groundCollision` $\rightarrow$ **`integrateGround`**: eliminates one round-trip for position/velocity.

Fusion keeps intermediate results in registers rather than writing to global memory and re-reading in a second kernel.

I. Spatial Hashing and CUB Radix Sort

Collision detection is reduced from $O(N^2)$ to $O(N \cdot k)$ via uniform-grid spatial hashing [4]. The hash function uses prime multiplication: $h = (c_x \cdot 73856093) \oplus (c_y \cdot 19349663) \oplus (c_z \cdot 83492791)$ with additional bit-mixing, masked to a compact table of size `nextPowerOf2`$(N \times 8)$. This reduces memory from $\sim$75 MB (full domain) to $\sim$256 KB.

CUB `DeviceRadixSort::SortPairs` sorts (hash, particleIndex) pairs with a pre-allocated temporary buffer and bit-count limited to $\lceil \log_2(\text{tableSize})/4 \rceil \times 4$ to minimise radix passes. A `clearPrevCellsKernel` clears only cells used in the previous frame (scatter-clear), avoiding a full-array `memset`.

J. Sorted Snowpack and Binary-Search Capture

The snowpack is static — sorting it once at initialisation by `posX` enables a host-side `std::lower_bound`/`upper_bound` each frame to find the interval $[\text{lo}, \text{hi})$ within capture distance. The kernel launches on this subset only, reducing threads from 10^6 to ∼500–2,000 (a 50–100× reduction). The host-side binary search costs ∼1 μs on the pinned sorted array.

K. Mega-Fused Kernel for Small N

With $N \leq 2048$, the standard pipeline requires ∼10 GPU commands per frame, adding ∼100–150 μs of pure launch overhead on the GTX 960M (each command incurs ∼10–15 μs driver overhead). A single **`shellPhysicsFusedKernel`** replaces the entire pipeline: gravity → damping → tether → brute-force $O(N^2)$ collision → Euler integration → ground collision. Forces **never transit through global memory** — they stay in registers throughout. At $N = 2048$, brute-force collision is 4×10^6 iterations (∼0.3 ms), still well within budget. Above 2048 the grid-based $O(N \cdot k)$ path becomes more efficient.

L. Pinned Memory for Asynchronous Copies

Nsight Systems profiling revealed 30–40 ms GPU stalls caused by `cudaMemcpyAsync` targeting **pageable** host memory — the driver silently inserts a device-wide synchronisation. All host buffers for async readback are now allocated with `cudaMallocHost` (pinned memory): capture counters, sorted `posX`, and trace buffers. Additionally, the capture count (int) and mass (float) are packed into a single 8-byte buffer, reducing per-frame GPU commands from 4 to 2 (∼20–30 μs saved).

M. Multi-Stream Pipeline

Physics and grid-build kernels run on dedicated non-blocking CUDA streams. `simStream` executes forces → collision → integration; `gridStream` builds the spatial hash in parallel with force computation. An additional third `traceStream` handles asynchronous device-to-host copies for optional trace snapshots. Fork/join is implemented via `cudaEventRecord`/`cudaStreamWaitEvent` with timing-disabled events to minimise overhead.

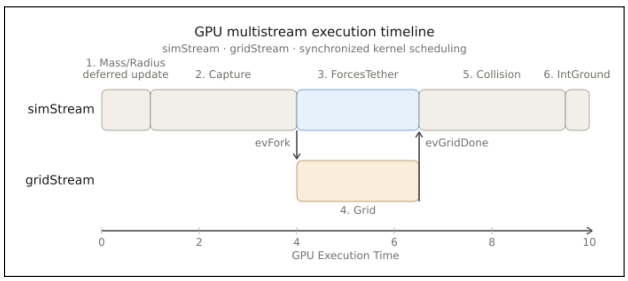

Fig. 3: Dual-stream pipeline. Stages 3 and 4 overlap on separate streams; stage 5 waits for both via a lightweight CUDA event.

N. Gated Event Recording

Each `cudaEventRecord()` call incurs ∼10–15 μs driver overhead. With up to 14 event records per frame (start + end for 7 timing pairs), this adds ∼110–165 μs per frame. Events are recorded only every `TIMING_INTERVAL = 64` frames (or when logging/rendering is active). The 63 non-profiled frames save ∼12 event records each. Synchronisation events (`evFork`, `evGridDone`) are always recorded since they are required for correctness.

O. ALU Optimizations

- **`rsqrtf` pattern**: $1/\sqrt{x}$ in a single SFU cycle (∼8 cycles), then `dist = dist2 * invDist` (1 FMA), saving one SFU op compared to `sqrtf + 1.0f/dist`.
- **Tangential friction**: squared-magnitude threshold (`vt2 > 1e-16f`) with `rsqrtf(vt2)` avoids a separate `sqrtf` + division.
- **Branchless clamping**: `fmaxf(f_spring - f_damp, 0.0f)` compiles to a single `FMNMX` instruction (1 cycle) on Maxwell.
- **Hash-based PRNG**: a murmurhash variant (6 integer ops) replaces `curand` in the capture kernel, eliminating 48 B/thread of global-memory state.
- **Quaternion rotation**: the Rodrigues formula uses 15 FMA operations versus 27 for a 3×3 matrix multiply, and avoids constructing a temporary matrix (9 registers).

P. Unity Build and Constant Sharing

All GPU source files (`grid.cu`, `simulation.cu`, `snowpack.cu`) are `#included` into a single translation unit `gpu_unity.cu`. This shares the `__constant__ SimParams d_params` declaration across all kernels without separate compilation, avoiding device-link overhead.

Q. Memory Hierarchy Summary

Table III summarises how each memory level is exploited.

TABLE III: Memory hierarchy utilisation.

Level	Usage	Size
Registers	Local vars, force accumulators	24–48/thread
`__constant__`	`SimParams` (broadcast)	∼152 B
Shared memory	Warp-cooperative tile	12.4 KB/blk
Read-only cache	`__ldg()` loads (immutable SoA fields)	HW-managed
L2 cache	Particle working set	1 MB / 256 KB
Global (GDDR5)	SoA arrays	$16N \times 4$ B

R. Holistic Optimization Strategy

The optimizations described above are not applied independently, but as an integrated strategy responding to Maxwell's specific constraints. Register pressure is minimized through constant memory (broadcasts), tiling (shared-memory reuse), and fusion (avoiding intermediate global writes). This architecture-aware methodology ensures that every optimization decision is motivated by profiling data and hardware-specific trade-offs, rather than generic best practices.

VI CPU Reference Implementations

Two CPU builds mirror the GPU physics pipeline, enabling direct speedup measurements against the same simulation logic. No rendering capabilities are included in these builds.

A. SISD Baseline

The scalar build (`AngrySanta_SISD`) is compiled with all compiler optimisations disabled (`-O0`, `/Od`), no inlining, no auto-vectorisation, and no loop unrolling. It serves as a strict performance floor for measuring both GPU and SIMD speedups. The simulation loop in `simulation_CPU.cpp` executes the same pipeline: capture → forces + tether → grid → collision → integrate + ground → core update.

B. SIMD + OpenMP

The optimized build (`AngrySanta_SIMD`) uses SSE2 intrinsics on x86-64 and NEON intrinsics on ARM64, plus the OpenMP directives for multi-threading. The SIMD implementation is portable across both platforms, with conditional compilation to include the appropriate headers and define platform-specific macros:

```
#if defined(__x86_64__) || defined(_M_X64)
  #include <immintrin.h>   // SSE / SSE2
  #define CPU_SIMD_X86 1
#elif defined(__aarch64__)
  #include <arm_neon.h>    // NEON
  #define CPU_SIMD_NEON 1
#endif
```

Each physics function processes 4 particles per iteration (128-bit SIMD lanes) with a scalar remainder loop for the $N \bmod 4$ tail. SoA arrays are allocated with 16-byte alignment (via platform-specific `_aligned_malloc` / `aligned_alloc` macros) to enable aligned packed loads (`_mm_load_ps` / `vld1q_f32`).

```
// In SIMD builds, allocate 16-byte aligned memory for
    packed loads/stores.
#ifdef SIMD
  #ifdef _MSC_VER
    #define ALLOC(bytes)  _aligned_malloc((bytes), 16)
    #define FREE(ptr)     _aligned_free(ptr)
  #else
    static inline void* simd_alloc_impl_(size_t bytes) {
      size_t aligned = (bytes + 15u) & ~(size_t)15u;
      return aligned_alloc(16, aligned > 0 ? aligned : 16);
    }
    #define ALLOC(bytes)  simd_alloc_impl_(bytes)
    #define FREE(ptr)     free(ptr)
  #endif
#else
  #define ALLOC(bytes)  malloc(bytes)
  #define FREE(ptr)     free(ptr)
#endif
```

When compiled with `-DHAS_OPENMP`, data-parallel loops use `#pragma omp parallel for schedule(static)` for multi-threaded distribution across CPU cores, applying to initialisation, force computation, and integration. The only algorithmic difference is no spatial hashing — collision detection remains $O(N^2)$ brute-force. This ensures that GPU speedup measurements isolate parallelization and architectural advantage rather than algorithmic improvements, and allows direct quantification of SIMD vs. GPU performance.

VII Experimental Results

A. Experimental Setup

TABLE IV: Hardware specifications of the two test platforms.

	Laptop (Windows)	**Jetson Nano (Linux)**
Host CPU	Intel i7-6700HQ	ARM Cortex-A57 (SoC)
CPU cores / threads	4 / 8	4 / 4
GPU	GeForce GTX 960M	Tegra X1
Compute capability	SM 5.0	SM 5.3
VRAM	4 GB	4 GB
SMs	5	1
CUDA cores / SM	128	128
Total CUDA cores	640	128
L2 cache	1 MB	256 KB
Memory	4 GB GDDR5	4 GB LPDDR4 (shared)
Memory bandwidth	∼30 GB/s	∼25.6 GB/s

B. GTX960M Per-Kernel Timing Physics Scenario

These scenarios use a small number of particles ($N = 10^5$) causing the simulation on GPU to remain in the fused regime.

TABLE V: Wet snow.
-

Kernel	**SISD (ms)**	**SIMD (ms)**	**GPU (ms)**
captureFromSnowpack	0.0589	0.0568	0.0960
forceTether	0.0091	0.0101	0.0000
shellGrid	0.0365	0.0335	0.0000
bruteForceCollision	0.0904	0.0184	0.0000
integrateGround	0.0055	0.0101	0.0000
shellPhysicsFused	0.0000	0.0000	0.0846
coreUpdate	0.0002	0.0001	0.0000
Overhead	0.0005	0.0004	0.0414
FPS	4975.2	7723.4	4503.9

TABLE VI: Dry snow.
–wetness-max 0.3 –stick-k1 3.0

Kernel	**SISD (ms)**	**SIMD (ms)**	**GPU (ms)**
captureFromSnowpack	0.0345	0.0413	0.0765
forceTether	0.0047	0.0083	0.0000
shellGrid	0.0170	0.0227	0.0000
bruteForceCollision	0.0394	0.0086	0.0000
integrateGround	0.0029	0.0082	0.0000
shellPhysicsFused	0.0000	0.0000	0.0583
coreUpdate	0.0001	0.0001	0.0000
Overhead	0.0005	0.0004	0.0338
FPS	10077.4	11174.3	5933.1

TABLE VII: Amplified avalanche feedback.
–stick-rboost 10.0

Kernel	**SISD (ms)**	**SIMD (ms)**	**GPU (ms)**
captureFromSnowpack	0.0508	0.0519	0.1108
forceTether	0.0102	0.0098	0.0000
shellGrid	0.0435	0.0369	0.0000
bruteForceCollision	0.1094	0.0243	0.0000
integrateGround	0.0062	0.0100	0.0000
shellPhysicsFused	0.0000	0.0000	0.1011
coreUpdate	0.0001	0.0001	0.0000
Overhead	0.0005	0.0004	0.0485
FPS	4529.0	7491.9	3839.4

TABLE VIII: High inter-particle friction.
–part-friction 0.6

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.0656	0.0932	0.0924
forceTether	0.0096	0.0275	0.0000
shellGrid	0.0394	0.0442	0.0000
bruteForceCollision	0.0934	0.0288	0.0000
integrateGround	0.0058	0.0148	0.0000
shellPhysicsFused	0.0000	0.0000	0.0887
coreUpdate	0.0002	0.0002	0.0000
Overhead	0.0007	0.0005	0.0440
FPS	4657.1	4783.6	4442.7

TABLE IX: Slope 45°.
–slope 45.0

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.0454	0.0641	0.1035
forceTether	0.0073	0.0135	0.0000
shellGrid	0.0274	0.0338	0.0000
bruteForceCollision	0.0618	0.0163	0.0000
integrateGround	0.0045	0.0138	0.0000
shellPhysicsFused	0.0000	0.0000	0.0716
coreUpdate	0.0002	0.0002	0.0000
Overhead	0.0007	0.0005	0.0431
FPS	6791.4	7035.4	4583.0

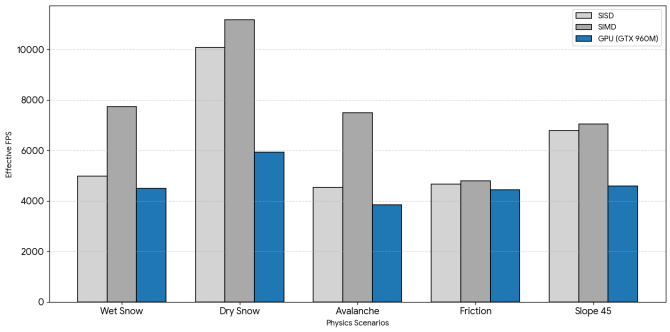

Fig. 4: FPS comparison across the 5 physics scenarios on the GTX960M.

At $N = 10^5$ the GTX960M consistently yields lower FPS than both CPU baselines across all five physics scenarios. The root cause is a structural overhead problem: the shell layer never grows beyond $\sim$2 048 particles at this scale, so the `shellPhysicsFused` kernel executes in a single thread-block and occupies only one of the five available SMs. The remaining GPU idle time is dominated by fixed costs—CUDA kernel launch latency ($\sim 5\,\mu$s per kernel), host-device event synchronization, and PCIe memory transfers—that together account for roughly **35–50%** of total frame time.
The i7-6700HQ, by contrast, executes the equivalent sequential physics loop with minimal IPC stalls, benefiting from its deep out-of-order pipeline and large L3 cache that keeps the 10^5-particle snowpack entirely in cache. SIMD further accelerates the CPU path through 4-wide AVX vectorisation of the per-particle force loop, reaching peak FPS in the dry-snow scenario (11 174 FPS) where the brute-force collision work is lightest.

These results highlight an important design principle: GPU parallelism only pays off once the workload is large enough to amortize launch and transfer overheads. The crossover point is explored in the next section and occurs between $N = 100{,}000$ and $N = 500{,}000$ on this platform.

C. Scaling with Particle Count

TABLE X: $N = 500{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.2293	0.0895
forceTether	0.0098	0.0143
shellGrid	0.1753	0.0126
neighborCollision	0.1601	0.0297
integrateGround	0.0104	0.0021
shellPhysicsFused	0.0000	0.1348
coreUpdate	0.0001	0.0000
Overhead	0.0008	0.0398
FPS	1706.6	3097.8

TABLE XI: $N = 1{,}000{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.6353	0.1038
forceTether	0.0192	0.0276
shellGrid	0.4268	0.0538
neighborCollision	0.6570	0.0837
integrateGround	0.0596	0.0039
shellPhysicsFused	0.0000	0.0904
coreUpdate	0.0002	0.0000
Overhead	0.0010	0.0528
FPS	555.8	2404.5

TABLE XII: $N = 5{,}000{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	3.9422	0.1653
forceTether	0.1069	0.0824
shellGrid	3.1953	0.1927
neighborCollision	8.7393	0.9272
integrateGround	0.1052	0.0212
shellPhysicsFused	0.0000	0.0322
coreUpdate	0.0004	0.0000
Overhead	0.0015	0.0833
FPS	62.1	664.7

TABLE XIII: $N = 10{,}000{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	7.5114	0.2210
forceTether	0.2245	0.1363
shellGrid	7.7466	0.2977
neighborCollision	26.8500	2.6099
integrateGround	0.1709	0.0484
shellPhysicsFused	0.0000	0.0192
coreUpdate	0.0004	0.0000
Overhead	0.0015	0.1031
FPS	23.5	291.1

TABLE XIV: GPU vs. SIMD CPU scaling with size N.

N	SIMD (FPS)	GPU (FPS)	SIMD (ms/f)	GPU (ms/f)	Speedup
500,000	1,706.6	3,097.8	0.586	0.323	1.8×
1,000,000	555.8	2,404.5	1.799	0.416	4.3×
5,000,000	62.1	664.7	16.10	1.505	10.7×
10,000,000	23.5	291.1	42.55	3.435	12.4×

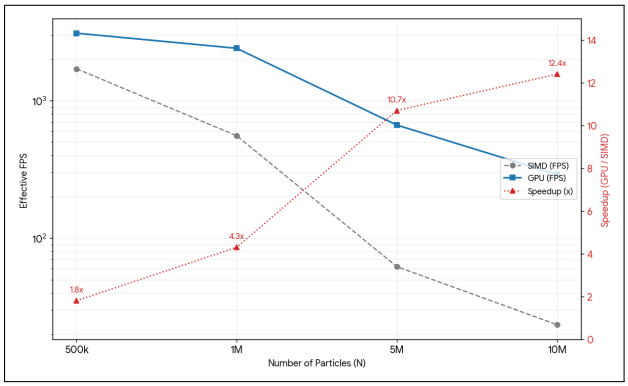

Fig. 5: Scaling: FPS SIMD CPU vs GTX960M.

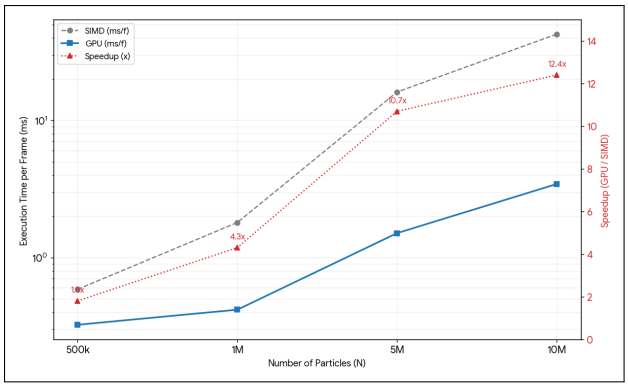

Fig. 6: Scaling: ms SIMD CPU vs GTX960M.

As N grows the GPU's parallel capture kernel and $O(N \cdot k)$ grid-based collision scale far better than the CPU's $O(N^2)$ brute-force, reaching a **12.4× speedup** at $N = 10^7$. Overhead grows modestly with N and never exceeds 3% of frame time at large scales, confirming that PCIe transfer cost is amortized at scale.

D. Jetson Nano Per-Kernel Timing Physics Scenario

These scenarios use a small number of particles ($N = 10^5$) causing the simulation on GPU to remain in the fused regime.

TABLE XV: Wet snow.
-

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.2554	0.1949	0.0329
forceTether	0.0354	0.0057	0.0000
shellGrid	0.1062	0.0327	0.0000
bruteForceCollision	0.3813	0.0628	0.0000
integrateGround	0.0237	0.0057	0.0000
shellPhysicsFused	0.0000	0.0000	0.1246
coreUpdate	0.0004	0.0003	0.0000
Overhead	0.0012	0.0007	0.0192
FPS	1244.3	3302.7	5660.1

TABLE XVI: Dry snow.
–wetness-max 0.3 –stick-k1 3.0

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.1497	0.1870	0.0468
forceTether	0.0164	0.0061	0.0000
shellGrid	0.0452	0.0222	0.0000
bruteForceCollision	0.1452	0.0248	0.0000
integrateGround	0.0112	0.0065	0.0000
shellPhysicsFused	0.0000	0.0000	0.1083
coreUpdate	0.0004	0.0003	0.0000
Overhead	0.0012	0.0006	0.0331
FPS	2707.5	4039.0	5314.7

TABLE XVII: Amplified avalanche feedback.
–stick-rboost 10.0

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.2489	0.2031	0.0498
forceTether	0.0420	0.0076	0.0000
shellGrid	0.1273	0.0389	0.0000
bruteForceCollision	0.4790	0.0719	0.0000
integrateGround	0.0285	0.0068	0.0000
shellPhysicsFused	0.0000	0.0000	0.1974
coreUpdate	0.0004	0.0003	0.0000
Overhead	0.0012	0.0007	0.0412
FPS	1078.4	3036.5	3466.9

TABLE XVIII: High inter-particle friction.
–part-friction 0.6

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.2557	0.2188	0.0507
forceTether	0.0351	0.0069	0.0000
shellGrid	0.1067	0.0354	0.0000
bruteForceCollision	0.3818	0.0597	0.0000
integrateGround	0.0236	0.0068	0.0000
shellPhysicsFused	0.0000	0.0000	0.1723
coreUpdate	0.0004	0.0003	0.0000
Overhead	0.0012	0.0007	0.0363
FPS	1242.9	3044.0	3854.4

TABLE XIX: Slope 45°.
–slope 45.0

Kernel	SISD (ms)	SIMD (ms)	GPU (ms)
captureFromSnowpack	0.1840	0.1892	0.0447
forceTether	0.0202	0.0066	0.0000
shellGrid	0.0573	0.0247	0.0000
bruteForceCollision	0.1794	0.0287	0.0000
integrateGround	0.0138	0.0070	0.0000
shellPhysicsFused	0.0000	0.0000	0.1334
coreUpdate	0.0004	0.0003	0.0000
Overhead	0.0012	0.0007	0.0291
FPS	2191.0	3890.7	4823.8

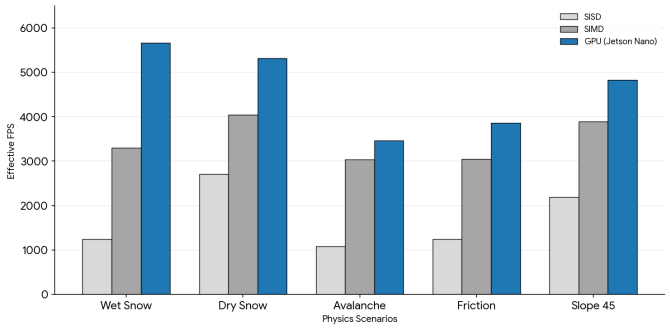

Fig. 7: FPS comparison across the 5 physics scenarios on the Jetson Nano.

At $N = 10^5$, the Jetson Nano GPU slightly outperforms both CPU baselines in all five physics scenarios—a stark contrast to the GTX 960M, which is slower than the CPU at this scale. The unified memory architecture eliminates PCIe transfer overhead entirely, so even a single SM delivers a consistent GPU advantage; kernel-launch and event overhead account for only 11–18% of GPU frame time vs. 35–50% on the GTX 960M.

E. Scaling with Particle Count

TABLE XX: $N = 500{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	1.5144	0.4723
forceTether	0.0311	0.0048
shellGrid	0.2603	0.0198
neighborCollision	0.7858	0.1176
integrateGround	0.0249	0.0044
shellPhysicsFused	0.0000	0.3468
coreUpdate	0.0004	0.0000
Overhead	0.0010	0.6080
FPS	382.0	635.4

TABLE XXI: $N = 1{,}000{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	3.0397	0.6675
forceTether	0.0601	0.0158
shellGrid	0.6560	0.1044
neighborCollision	2.5563	4.1633
integrateGround	0.0582	0.0127
shellPhysicsFused	0.0000	0.2029
coreUpdate	0.0005	0.0000
Overhead	0.0012	0.9778
FPS	156.9	162.8

TABLE XXII: $N = 5{,}000{,}000$.

Kernel	SIMD (ms)	GPU (ms)
captureFromSnowpack	14.9198	0.7978
forceTether	0.3281	0.0897
shellGrid	4.7699	0.5537
neighborCollision	36.9670	174.8856
integrateGround	0.3268	0.0943
shellPhysicsFused	0.0000	0.0774
coreUpdate	0.0005	0.0000
Overhead	0.0016	1.4548
FPS	17.4	5.6

N = 10M scaling was not possible due to GPU out-of-memory errors and a catastrophic performance drop on both CPU and GPU.

TABLE XXIII: GPU vs. SIMD CPU scaling with size N.

N	SIMD (FPS)	GPU (FPS)	SIMD (ms/f)	GPU (ms/f)	Speedup
500,000	382.0	635.4	2.618	1.574	1.7×
1,000,000	156.9	162.8	6.372	6.144	1.0×
5,000,000	17.4	5.6	57.314	177.953	0.3×
10,000,000	-	-	-	-	-

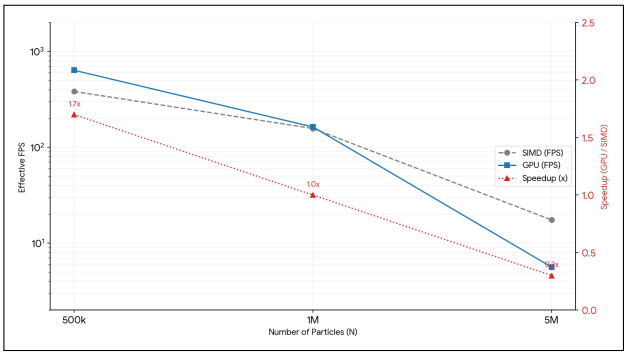

Fig. 8: Scaling: FPS SIMD CPU vs Jetson Nano.

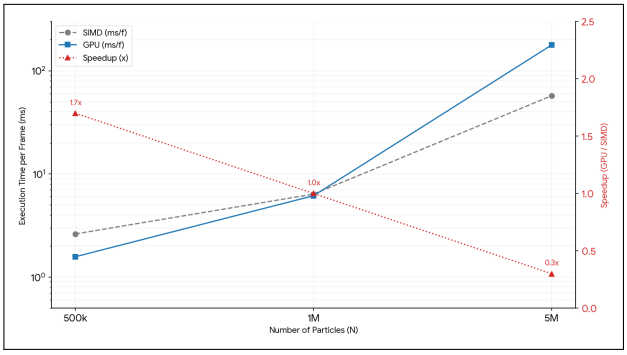

Fig. 9: Scaling: ms SIMD CPU vs Jetson Nano.

The Jetson Nano's single Maxwell SM (128 cores, 2048 max threads) is the fundamental bottleneck at large N. With ∼72K shell particles and a single SM limited to 2 048 concurrent threads, the collision kernel serialises into ∼36 sequential warp waves, each performing $O(k)$ neighbour lookups. This causes super-linear growth in execution time: the shell grows only 6.7× from $N = 10^6$ to $N = 5{\times}10^6$, yet GPU collision time grows 42× (4.16 ms → 174.9 ms), while the 4-core SIMD CPU grows only 14.5× on the same workload.

In contrast, the 4-core multi-threaded SIMD CPU scales far more gracefully under the same workload.

A multi-SM desktop GPU—such as the GTX 960M with 5 SMs and 640 cores effectively mitigates this bottleneck by distributing the massive wave volume across multiple physical processing blocks, maintaining high throughput even in heavily saturated particle regimes.

VIII Conclusion

We presented a GPU-accelerated Discrete Element Method simulation of a growing snowball rolling down an inclined slope, targeting NVIDIA Maxwell hardware (GTX 960M and Jetson Nano). The key takeaways are:

1) **Architecture-aware optimisation matters.** On resource-constrained Maxwell GPUs, each register, each kilobyte of shared memory, and each kernel launch has measurable impact. The warp-cooperative tiling design, zero-spill register allocation, and gated event recording are all responses to specific Maxwell bottlenecks (no hardware FP32 atomics, 48 KB shared memory limit, high per-command driver overhead).
2) **Adaptive fusion pays off.** The dual-path pipeline (mega-fused for $N \leq 2048$, grid-based for $N > 2048$) eliminates launch overhead during the initial growth phase while seamlessly transitioning to the scalable $O(N \cdot k)$ path as the particle count increases.
3) **Zero spill trumps 100% occupancy.** Accepting 62.5–75% occupancy in exchange for keeping all variables in physical registers avoids DRAM-backed spill costs ($\sim$300 cycles per access) and yields better real-world performance than forcing higher occupancy through aggressive register limiting.

A. Future Work

Several extensions could be developed:

- **Multi-GPU / domain decomposition**: partitioning the spatial grid across multiple GPUs via halo exchange for simulations with very large shell particle counts (refer to Attachment 1).
- **Persistent-contact DEM**: tracking tangential spring history across frames for more accurate friction modelling, at the cost of additional per-pair state.
- **Volta+ optimisations**: on SM 7.0+, hardware `atomicAdd(float)` eliminates CAS loops, potentially making symmetric collision viable again; cooperative groups could replace warp-level synchronisation.
- **3D terrain**: replacing the infinite plane with a height-map or mesh-based terrain for realistic mountain geometry.

Project Code

The complete source code for **Angry Santa** is available on GitHub: https://github.com/politos336145/GPU-Programming-2025-2026

Acknowledgments

The authors thank the GPU Programming course staff at Politecnico di Torino for guidance and access to the Jetson Nano development kits.

References

[1] P. A. Cundall and O. D. L. Strack, "A discrete numerical model for granular assemblies," *Géotechnique*, vol. 29, no. 1, pp. 47–65, 1979.
[2] S. Green, "Particle simulation using CUDA," in *GPU Gems 3*. NVIDIA / Addison-Wesley, 2010.
[3] T. Pöschel and T. Schwager, *Computational Granular Dynamics: Models and Algorithms*. Springer, 2005.
[4] M. Teschner, B. Heidelberger, M. Müller, D. Pomerantes, and M. H. Gross, "Optimized spatial hashing for collision detection of deformable objects," in *Proc. Vision, Modeling, Visualization (VMV)*, 2003, pp. 47–54.
[5] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
[6] ——, *CUDA C++ Best Practices Guide*, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
[7] ——, *CUDA Toolkit Documentation*, 2024. [Online]. Available: https://docs.nvidia.com/cuda/
[8] ——, *NVIDIA Nsight Systems & Compute*, 2024. [Online]. Available: https://developer.nvidia.com/tools-overview
[9] ——, *CUB Library*, 2024. [Online]. Available: https://nvidia.github.io/cccl/unstable/cub/index.html
[10] Khronos Group, *Vulkan API Specification*, 2024. [Online]. Available: https://vulkan.lunarg.com/sdk/home
[11] Omar Cornut, *Dear Imgui API Reference*, 2024. [Online]. Available: https://github.com/ocornut/imgui